

PostgreSQL handles the following NoSQL data types:

- hstore
- xml
- json/jsonb

We will now talk about hstore and json.

The hstore data type

hstore was the first NoSQL data type that was implemented in PostgreSQL. This data type is used for storing key-value pairs in a single value. Before working with the hstore data type, we need to enable the hstore extension on our server:

```
forumdb=> create extension hstore ;  
CREATE EXTENSION
```

Let's look at how we can use the hstore data type with an example. Suppose that we want to show all posts with their usernames and their categories:

```
forumdb=> select p.pk,p.title,u.username,c.title as category  
from posts p  
inner join users u on p.author=u.pk  
left join categories c on p.category=c.pk  
order by 1;  
-[ RECORD 1 ]-----  
pk          | 5  
title       | Indexing PostgreSQL  
username    | luca_ferrari  
category    | Database  
-[ RECORD 2 ]-----  
pk          | 6  
title       | Indexing Mysql  
username    | luca_ferrari  
category    | Database  
-[ RECORD 3 ]-----  
pk          | 7  
title       | A view of Data types in C++  
username    | enrico_pirozzi  
category    | Programming Languages
```

Suppose now that the table's posts, users, and categories are huge tables and we would like to store all the information about usernames and categories in a single field stored inside the posts table. If we could do this, we would no longer need to join three huge tables. In this case, hstore can help us:

```

forumdb=> select p.pk,p.title,hstore(ARRAY['username',u.
username,'category',c.title]) as options
from posts p
inner join users u on p.author=u.pk
left join categories c on p.category=c.pk
order by 1;
-[ RECORD 1 ]-----
pk      | 5
title   | Indexing PostgreSQL
options | "category"=>"Database", "username"=>"luca_ferrari"
-[ RECORD 2 ]-----
pk      | 6
title   | Indexing Mysql
options | "category"=>"Database", "username"=>"luca_ferrari"
-[ RECORD 3 ]-----
pk      | 7
title   | A view of Data types in C++
options | "category"=>"Programming Languages", "username"=>"enrico_
pirozzi"

```

The preceding query first puts in an array the values of the username and category fields, and then transforms them into hstore. Now, if we want to store the data in a new table called posts_options, we have to perform something like the following:

```

forumdb=> create table posts_options as
select p.pk,p.title,hstore(ARRAY['username',u.username,'category',c.
title]) as options
from posts p
inner join users u on p.author=u.pk
left join categories c on p.category=c.pk
order by 1;
SELECT 3

```

We now have a new table with the following structure:

```
forumdb=> \d posts_options
          Table "forum.posts_options"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 pk      | integer |           |          |
 title   | text    |           |          |
 options | hstore  |           |          |
```

Next, suppose that we want to search for all the records that have `category = 'Database'`. We would have to execute the following:

```
forumdb=> select * from posts_options where options->'category'
='Database';
-[ RECORD 1 ]-----
 pk      | 5
 title   | Indexing PostgreSQL
 options | "category"=>"Database", "username"=>"luca_ferrari"
-[ RECORD 2 ]-----
 pk      | 6
 title   | Indexing Mysql
 options | "category"=>"Database", "username"=>"luca_ferrari"
```

Since `hstore`, as well as the `json/jsonb` data types, is not a structured data type, we can insert any other key value without defining it first – for example, we can do this:

```
forumdb=> insert into posts_options (pk,title,options) values (7,'my last
post','"enabled"=>"false"') ;
INSERT 0 1
```

The result of the selection on the whole table will be the following:

```
forumdb=> select * from posts_options;
-[ RECORD 1 ]-----
 pk      | 5
 title   | Indexing PostgreSQL
 options | "category"=>"Database", "username"=>"luca_ferrari"
-[ RECORD 2 ]-----
 pk      | 6
```

```

title   | Indexing Mysql
options | "category"=>"Database", "username"=>"luca_ferrari"
-[ RECORD 3 ]-----
pk      | 7
title   | A view of Data types in C++
options | "category"=>"Programming Languages", "username"=>"enrico_
pirozzi"
-[ RECORD 4 ]-----
pk      | 7
title   | my last post
options | "enabled"=>"false"

```

As we said at the beginning of this section, NoSQL is not the subject of this book, but it is worth briefly going over it. For further information about the hstore data type, please refer to the official documentation at <https://www.postgresql.org/docs/current/hstore.html>.

The JSON data type

In this section, we'll take a brief look at the JSON data type. **JSON** stands for **JavaScript Object Notation**. JSON is an open standard format, and it is formed of key-value pairs. PostgreSQL supports the JSON data type natively. It provides many functions and operators used for manipulating JSON data. PostgreSQL, in addition to the json data type, also supports the jsonb data type. The difference between these two data types is that the first is internally represented as text, whereas the second is internally represented in a binary and indexable manner. Let's look at how we can use the json/jsonb data types with an example.

Suppose that we want to show all the posts and tags that we have in our forumdb database. Working in a classic relational SQL way, we should write something like the following:

```

forumdb=> \x
Expanded display is off.
forumdb=> select p.pk,p.title,t.tag
from posts p
left join j_posts_tags jpt on p.pk=jpt.post_pk
left join tags t on jpt.tag_pk=t.pk
order by 1;
pk | title | tag
----+-----+-----

```

```

5 | Indexing PostgreSQL      | Operating Systems
5 | Indexing PostgreSQL      | Database
6 | Indexing Mysql           | Database
6 | Indexing Mysql           | Operating Systems
7 | A view of Data types in C++ | Database
(5 rows)

```

Suppose now that we want to have a result like the following:

pk	title	tag
5	Indexing PostgreSQL	Operating Systems,Database
6	Indexing PostgreSQL	Database,Operating Systems
7	A view of Data types in C++	Database

In a relational way, we have to aggregate data using the first two fields and perform something like the following:

```

forumdb=> \x
Expanded display is on.
forumdb=> select p.pk,p.title,string_agg(t.tag,',') as tag
from posts p
left join j_posts_tags jpt on p.pk=jpt.post_pk
left join tags t on jpt.tag_pk=t.pk
group by 1,2
order by 1;
-[ RECORD 1 ]-----
pk      | 5
title   | Indexing PostgreSQL
tag      | Operating Systems,Database
-[ RECORD 2 ]-----
pk      | 6
title   | Indexing Mysql
tag      | Database,Operating Systems
-[ RECORD 3 ]-----
pk      | 7

```

```

title | A view of  Data types in C++
tag   | Database

```

Now, imagine that we want to generate a simple JSON structure; we would execute the following query:

```

forumdb=> select row_to_json(q) as json_data from (
  select p.pk,p.title,string_agg(t.tag,',') as tag
  from posts p
  left join j_posts_tags jpt on p.pk=jpt.post_pk
  left join tags t on jpt.tag_pk=t.pk
  group by 1,2 order by 1) Q;
-[ RECORD 1 ]-----
json_data | {"pk":5,"title":"Indexing PostgreSQL","tag":"Operating
Systems,Database"}
-[ RECORD 2 ]-----
json_data | {"pk":6,"title":"Indexing Mysql","tag":"Database,Operating
Systems"}
-[ RECORD 3 ]-----
json_data | {"pk":7,"title":"A view of  Data types in
C++","tag":"Database"}

```

As we can see, with a simple query, it is possible to switch from a classic SQL representation to a NoSQL representation. Now, let's create a new table called `post_json`. This table will have only one jsonb field, called `jsondata`:

```

forumdb=> create table post_json (jsondata jsonb);
CREATE TABLE
forumdb=> \d post_json
          Table "forum.post_json"
   Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
jsondata | jsonb  |           |          |

```

Now, let's insert some data into the `post_json` table:

```

forumdb=> insert into post_json(jsondata)
select row_to_json(q) as json_data from (
  select p.pk,p.title,string_agg(t.tag,',') as tag
  from posts p

```

```

left join j_posts_tags jpt on p.pk=jpt.post_pk
left join tags t on jpt.tag_pk=t.pk
group by 1,2 order by 1) Q;
INSERT 0 3

```

Now, the post_json table has the following records:

```

forumdb=> select jsonb_pretty(jsontdata) from post_json;
-[ RECORD 1 ]+-----
jsonb_pretty | {                                +
              |   "pk": 5,                      +
              |   "tag": "Operating Systems,Database", +
              |   "title": "Indexing PostgreSQL"      +
              | }                                +
-[ RECORD 2 ]+-----
jsonb_pretty | {                                +
              |   "pk": 6,                      +
              |   "tag": "Database,Operating Systems", +
              |   "title": "Indexing Mysql"           +
              | }                                +
-[ RECORD 3 ]+-----
jsonb_pretty | {                                +
              |   "pk": 7,                      +
              |   "tag": "Database",              +
              |   "title": "A view of  Data types in C++" +
              | }                                +

```

If we wanted to search for all data that has tag = "Database", we could use the @> jsonb operator. This operator checks whether the left JSON value contains the right JSON path/value entries at the top level; the following query makes this search possible:

```

forumdb=> select jsonb_pretty(jsontdata) from post_json where jsontdata @>
'{"tag":"Database"}';

-[ RECORD 1 ]+-----
jsonb_pretty | {                                +
              |   "pk": 7,                      +
              |   "tag": "Database",            +

```

```
|      "title": "A view of  Data types in C++"+  
|  }
```

What we have just written is just a small taste of what can be done through the NoSQL data model. JSON is widely used when working with large tables and when a data structure is needed that minimizes the number of joins to be done during the research phase. A detailed discussion of the NoSQL world is beyond the scope of this book, but we wanted to describe briefly how powerful PostgreSQL is in the approach to unstructured data as well. For more information, please look at the official documentation at <https://www.postgresql.org/docs/current/functions-json.html>.

After understanding what data types are and which data types can be used in PostgreSQL, in the next section, we will see how to use data types within functions.

Exploring functions and languages

PostgreSQL is capable of executing server-side code. There are many ways to provide PostgreSQL with the code to be executed. For example, the user can create functions in different programming languages. The main languages supported by PostgreSQL are as follows:

- SQL
- PL/pgSQL
- C

These listed languages are the built-in languages; there are also other languages that PostgreSQL can manage, but before using them, we need to install them on our system. Some of these other supported languages are as follows:

- PL/Python
- PL/Perl
- PL/tcl
- PL/Java

In this section, we'll talk about SQL and PL/pgSQL functions.

Functions

The command structure with which a function is defined is as follows:

```
CREATE FUNCTION function_name(p1 type, p2 type,p3 type, ....., pn type)  
  RETURNS type AS
```

```
BEGIN
  -- function logic
END;
LANGUAGE language_name
```

The following steps always apply to any type of function we want to create:

1. Specify the name of the function after the `CREATE FUNCTION` keywords.
2. Make a list of parameters separated by commas.
3. Specify the return data type after the `RETURNS` keyword.
4. For the PL/pgSQL language, put some code between the `BEGIN` and `END` blocks.
5. For the PL/pgSQL language, the function has to end with the `END` keyword followed by a semicolon.
6. Define the language in which the function was written – for example, `sql` or `plpgsql`, `plperl`, `plpython`, and so on.

This is the basic scheme to which we will refer later in the chapter; this scheme may have small variations in some specific cases.

SQL functions

SQL functions are the easiest way to write functions in PostgreSQL, and we can use any SQL command inside them.

Basic functions

This section will show how to take your first steps into the SQL functions world. For example, the following function carries out a sum between two numbers:

```
forumbd=> CREATE OR REPLACE FUNCTION my_sum(x integer, y integer) RETURNS
integer AS $$
  SELECT x + y;
$$ LANGUAGE SQL;
CREATE FUNCTION

forumbd=> select my_sum(1,2);
my_sum
-----
```

```

      3
(1 row)

```

As we can see in the preceding example, the code function is placed between \$\$; we can consider \$\$ as labels. The function can be called using the SELECT statement without using any FROM clauses. The arguments of a SQL function can be referenced in the function body using either numbers (the old way) or their names (the new way). For example, we could write the same function in this way:

```

CREATE OR REPLACE FUNCTION my_sum(integer, integer) RETURNS integer AS $$
    SELECT $1 + $2;
$$ LANGUAGE SQL;

```

In the preceding function, we can see the old way to reference the parameter inside the function. In the old way, the parameters were referenced positionally, so the value \$1 corresponds to the first parameter of the function, \$2 to the second, and so on. In the code of the SQL functions, we can use all the SQL commands, including those seen in previous chapters.

SQL functions returning a set of elements

In this section, we will look at how to make a SQL function that returns a result set of a data type. For example, suppose that we want to write a function that takes p_title as a parameter and deletes all the records that have title=p_title, as well as returning all the keys of the deleted records. The following function would make this possible:

```

forumdb=> CREATE OR REPLACE FUNCTION delete_posts(p_title text) returns
setof integer as $$
delete from posts where title=p_title returning pk;
$$
LANGUAGE SQL;
CREATE FUNCTION

```

This is the situation before we called the delete_posts function:

```

forumdb=> select pk,title from posts order by pk;
 pk |          title
----+-----
  5 | Indexing PostgreSQL
  6 | Indexing Mysql
  7 | A view of  Data types in C++
(3 rows)

```